1-D Unsteady Scalar Transport

ME 7751 Homework 3

Toby Viet Nguyen

February 26, 2026

Scripts Included

`scalar_transport.py`, `plot_csv.py`, `part2.sh`, `part3.sh`, `part4.sh`, `part5.sh`, `part7.sh`.

Usage Instructions

All Bash scripts call functionality to `scalar_transport.py` and `plot_csv.py`. Run any script to regenerate all the images used for its corresponding part. Run `python scalar_transport.py -h` to see available command-line arguments.

Problem 1

Problem Statement

Solve numerically the one dimensional unsteady transport equation:

$$\frac{\partial \phi}{\partial t} + U\frac{\partial \phi}{\partial x} = \frac{\partial}{\partial x}\left(\Gamma \frac{\partial \phi}{\partial x}\right) \tag{1}$$

Here, U is the advection speed and Γ is the diffusion coefficient. The domain size is $L = 40$ with $5 \leq x \leq 45$. Boundary conditions are $\phi(5, t) = 0$ and $\phi(45, t) = 0$. The initial condition at $t = 10$ is $\phi(x, 10) = (0.4\pi)^{-0.5} \exp\left(-2.5(x-10)^2\right)$.

Consider two cases:

1. $U = 1$ and $\Gamma = 0.01$ with the analytical solution given by:

$$\phi(x, t) = (4\pi\Gamma t)^{-0.5} \exp\left(\frac{-(x - Ut)^2}{4\Gamma t}\right) \tag{2}$$

2. $U = 1$ and $\Gamma = 0$ with the analytical solution given by:

$$\phi(x, t) = (0.4\pi)^{-0.5} \exp\left(-2.5(x - Ut)^2\right) \tag{3}$$

1. Deriving Finite Difference Schemes

To derive the finite difference schemes for the governing equation (1), we first need to discretize the spatial and temporal domains. We discretize the spatial domain into N equally spaced points with spacing Δx. Let ϕ_i^n denote the numerical approximation of ϕ at spatial point i, $0 \leq i \leq N-1$, and time step n. The time step size is denoted by Δt.

Using central differences for the spatial derivatives and a forward difference for the time derivative, we can derive the finite difference scheme for (1). The discretized form of the equation is:

$$\frac{\phi_i^{n+1} - \phi_i^n}{\Delta t} + U\frac{\phi_{i+1}^n - \phi_{i-1}^n}{2\Delta x} = \Gamma\frac{\phi_{i+1}^n - 2\phi_i^n + \phi_{i-1}^n}{\Delta x^2} \tag{4}$$

Rearranging the equation to solve for ϕ_i^{n+1} gives us the explicit scheme:

$$\phi_i^{n+1} = \left(\frac{c}{2} + d\right)\phi_{i-1}^n + (1-2d)\,\phi_i^n + \left(-\frac{c}{2} + d\right)\phi_{i+1}^n \tag{5}$$

Here, $c = \frac{U\Delta t}{\Delta x}$ and $d = \frac{\Gamma\Delta t}{\Delta x^2}$.

We can replace n with $n+1$ in the advection and diffusion terms in (4) to derive the implicit scheme:

$$\frac{\phi_i^{n+1} - \phi_i^n}{\Delta t} + U\frac{\phi_{i+1}^{n+1} - \phi_{i-1}^{n+1}}{2\Delta x} = \Gamma\frac{\phi_{i+1}^{n+1} - 2\phi_i^{n+1} + \phi_{i-1}^{n+1}}{\Delta x^2} \tag{6}$$

This scheme can be rewritten as a matrix-vector equation, which can be solved using the tridiagonal matrix algorithm (TDMA) at each time step:

$$\left(-\frac{c}{2} - d\right)\phi_{i-1}^{n+1} + (1+2d)\phi_i^{n+1} + \left(\frac{c}{2} - d\right)\phi_{i+1}^{n+1} = \phi_i^n \tag{7}$$

$$\begin{bmatrix} 1+2d & \frac{c}{2}-d & 0 & \cdots & 0 \\ -\frac{c}{2}-d & 1+2d & \frac{c}{2}-d & \cdots & 0 \\ 0 & -\frac{c}{2}-d & 1+2d & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1+2d \end{bmatrix} \begin{Bmatrix} \phi_0^{n+1} \\ \phi_1^{n+1} \\ \phi_2^{n+1} \\ \vdots \\ \phi_{N-1}^{n+1} \end{Bmatrix} = \begin{Bmatrix} \phi_0^n \\ \phi_1^n \\ \phi_2^n \\ \vdots \\ \phi_{N-1}^n \end{Bmatrix} \tag{8}$$

In short, we can derive the Crank-Nicolson scheme by taking the average of the convection and diffusion terms at time steps n and $n+1$. This can be expressed as:

$$\begin{aligned} \phi_i^{n+1} = \phi_i^n &+ \frac{\Delta t}{2}\left[-\frac{U}{2\Delta x}(\phi_{i+1}^n - \phi_{i-1}^n) + \frac{\Gamma}{\Delta x^2}(\phi_{i+1}^n - 2\phi_i^n + \phi_{i-1}^n)\right] \\ &+ \frac{\Delta t}{2}\left[-\frac{U}{2\Delta x}(\phi_{i+1}^{n+1} - \phi_{i-1}^{n+1}) + \frac{\Gamma}{\Delta x^2}(\phi_{i+1}^{n+1} - 2\phi_i^{n+1} + \phi_{i-1}^{n+1})\right] \end{aligned} \tag{9}$$

We can rearrange this equation to express it in the form of a matrix-vector equation similar to (8), which can also be solved using TDMA.

$$\left(-\frac{c}{4} - \frac{d}{2}\right)\phi_{i-1}^{n+1} + (1+d)\phi_i^{n+1} + \left(\frac{c}{4} - \frac{d}{2}\right)\phi_{i+1}^{n+1} = \psi_i^n \tag{10}$$

Here, we use the substitution $\psi_i^n = \left(\frac{c}{4} + \frac{d}{2}\right)\phi_{i-1}^n + (1-d)\phi_i^n + \left(-\frac{c}{4} + \frac{d}{2}\right)\phi_i^{n+1}$ to express the right-hand side of the equation in a more compact form. The resulting matrix-vector equation for the Crank-Nicolson scheme is:

$$\begin{bmatrix} 1+d & \frac{c}{4}-\frac{d}{2} & 0 & \cdots & 0 \\ -\frac{c}{4}-\frac{d}{2} & 1+d & \frac{c}{4}-\frac{d}{2} & \cdots & 0 \\ 0 & -\frac{c}{4}-\frac{d}{2} & 1+d & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1+d \end{bmatrix} \begin{Bmatrix} \phi_0^{n+1} \\ \phi_1^{n+1} \\ \phi_2^{n+1} \\ \vdots \\ \phi_{N-1}^{n+1} \end{Bmatrix} = \begin{Bmatrix} \psi_0^n \\ \psi_1^n \\ \psi_2^n \\ \vdots \\ \psi_{N-1}^n \end{Bmatrix} \tag{11}$$

It is worth noting that in all of these schemes, we assumed $\phi_{-1}^n = \phi_N^n = 0$ since they are not defined in the problem statement, allowing us to represent the schemes in these compact forms.

To account for the boundary conditions, we must set $\phi_0^n = \phi_{N-1}^n = 0$ for all time steps n. Additionally, we must modify the first and last rows of the coefficient matrices in (8) and (11) to enforce these boundary conditions.

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ -\frac{c}{2} - d & 1 + 2d & \frac{c}{2} - d & \cdots & 0 \\ 0 & -\frac{c}{2} - d & 1 + 2d & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix} \tag{12}$$

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ -\frac{c}{4} - \frac{d}{2} & 1 + d & \frac{c}{4} - \frac{d}{2} & \cdots & 0 \\ 0 & -\frac{c}{4} - \frac{d}{2} & 1 + d & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix} \tag{13}$$

Here, (12) and (13) represent the modified coefficient matrices for the implicit and Crank-Nicolson schemes, respectively, after enforcing the boundary conditions.

2. Effect of Diffusion on Stability

In this initial test, we compare the results of case 1 ($U = 1, \Gamma = 0.01$) and case 2 ($U = 1, \Gamma = 0$) using the three schemes. $N = 501$ spatial points and $\Delta t = 0.01$ for all cases and schemes. Figure 1 shows the numerical and analytical solutions of all cases and schemes at $t = 20, 30, 40$.

The most notable difference between the two cases is that the numerical solutions in case 2 are more oscillatory than those in case 1, especially for the explicit scheme. This is because the diffusion term in case 1 has a stabilizing effect on the numerical solution, while the absence of diffusion in case 2 allows for more oscillations to develop. The implicit and Crank-Nicolson schemes are less affected by this instability compared to the explicit scheme, but they still show some oscillations in case 2. Overall, the results demonstrate that diffusion has a significant effect on the stability of the numerical solution, and the choice of numerical scheme can also impact the stability and accuracy of the solution, especially in cases with low or zero diffusion.

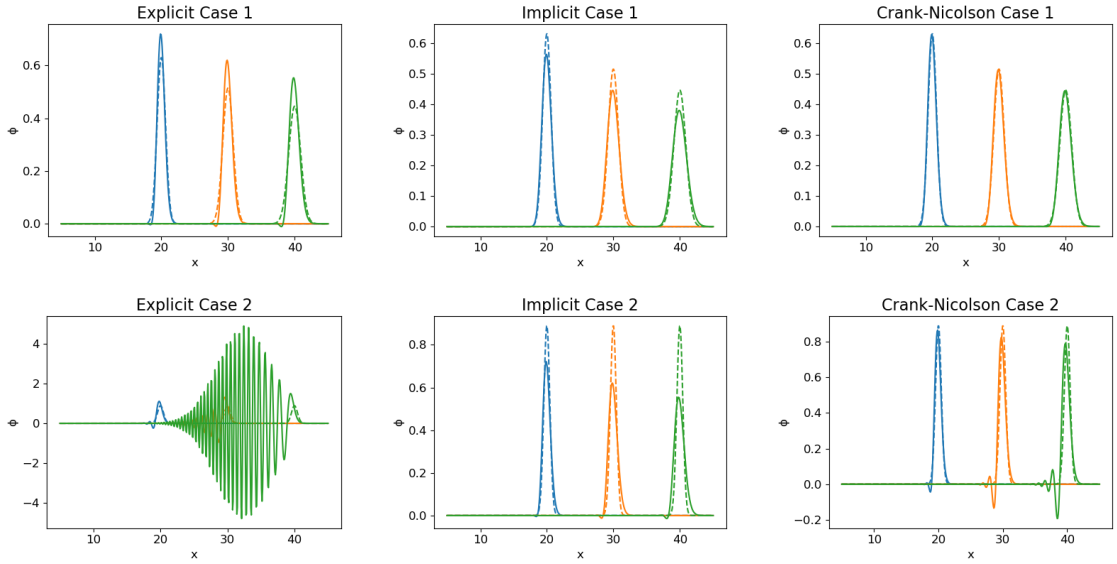

Figure 1: Comparison of the explicit, implicit, and Crank-Nicolson schemes in cases 1 and 2. Numerical solutions (solid lines) and analytical solutions (dashed lines) are plotted at $t = 20$ (blue lines), $t = 30$ (orange lines), and $t = 40$ (green lines).

3. Effect of Scheme on Accuracy

We now compare the results of the three schemes for case 1 ($U = 1, \Gamma = 0.01$) using $N = 501$ spatial points and $\Delta t = 0.01$. Figure 2 shows the numerical and analytical solutions of all schemes at $x = 15, 25$.

While it is difficult to visually distinguish the numerical solutions from the analytical solutions in all three schemes, we can see that the explicit scheme overestimates the analytical solution while the implicit scheme underestimates the analytical solution. The Crank-Nicolson scheme, on the other hand, provides a much closer approximation to the analytical solution compared to the other two schemes. This is because the Crank-Nicolson scheme is a second-order accurate method in both space and time, while the explicit and implicit schemes are only first-order accurate in time. More intuitively, the Crank-Nicolson is an average of the explicit and implicit schemes, which allows it to capture the behavior of the solution more accurately by balancing the errors from both schemes.

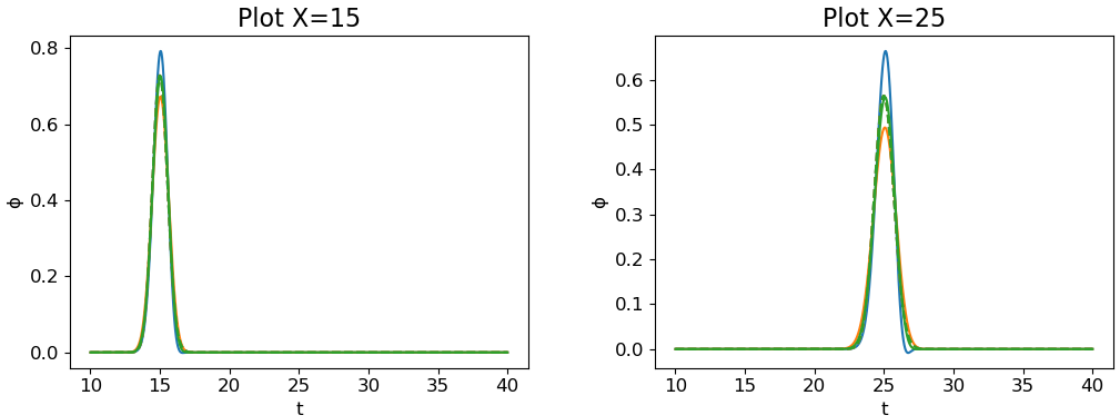

Figure 2: Numerical solutions (solid lines) and analytical solutions (dashed lines) are plotted (with respect to time) at $x = 15, 25$ using the explicit scheme (blue lines), implicit scheme (orange lines), and Crank-Nicolson scheme (green lines).

4. Analysis of Spatial Convergence

We can investigate the spatial convergence of the three schemes by computing the mean absolute error (MAE) between the numerical and analytical solutions at $t = 40$ for different values of N. Figure 3 shows a log-log plot of MAE versus Δx for the Crank-Nicolson scheme, using $N = 401, 801, 1601, 3201, 6401$. MAE is computed according to:

$$\text{MAE} = \frac{1}{N} \sum_{i=0}^{N-1} |\phi_i^{\text{numerical}} - \phi_i^{\text{analytical}}| \tag{14}$$

The plot shows that the MAE decreases as Δx decreases, indicating that the numerical solution is converging to the analytical solution as the spatial resolution increases. The slope of the line in the log-log plot is approximately 2, which confirms that the Crank-Nicolson scheme is second-order accurate in space. This means that as we halve Δx, the error decreases by a factor of approximately $2^2 = 4$. This convergence behavior is consistent with the theoretical properties of the Crank-Nicolson scheme.

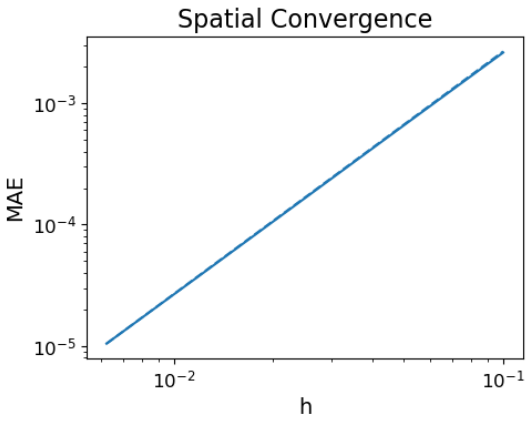

Figure 3: Spatial convergence plot of the Crank-Nicolson scheme. The slope of the line is approximately 2, indicated by the dashed line.

5. Analysis of Temporal Convergence

We can also investigate the temporal convergence of the three schemes by computing the MAE between the numerical and analytical solutions at $t = 40$ for different values of Δt. Figure 4 shows a log-log plot of MAE versus Δt for the three schemes, using $\Delta t = 0.0125, 0.00625, 0.003125, 0.0015625, 0.00078125$.

The plot shows that the MAE decreases as Δt decreases, indicating that the numerical solution is converging to the analytical solution as the temporal resolution increases. The slope of the line in the log-log plot is approximately 0.7 for the explicit scheme (dashed, blue), 0.6 for the implicit scheme (dashed, orange), and 0.007 for the Crank-Nicolson scheme (dashed, green). These don't correlate well with the theoretical properties of the schemes, which predict that the explicit and implicit schemes should be first-order accurate in time and the Crank-Nicolson scheme should be second-order accurate in time. The discrepancy could be due to several factors, such as numerical errors, the choice of Δt values, or the specific implementation of the schemes. Further investigation would be needed to understand the reasons for this discrepancy and to confirm the expected convergence rates.

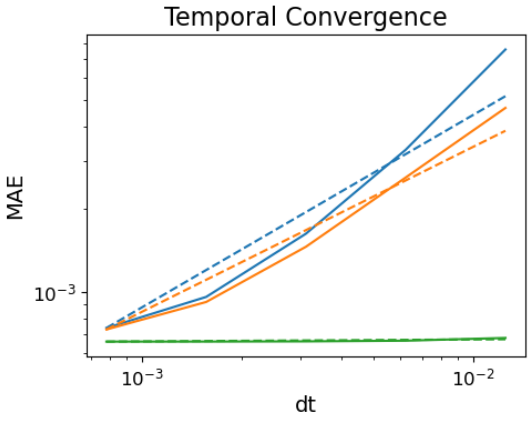

Figure 4: Temporal convergence plot of the explicit scheme (solid, blue), implicit (solid, orange), and Crank-Nicolson (solid, green). The slopes of the dashed lines are 0.7 (dashed, blue), 0.6 (dashed, orange), and 0.007 (dashed, green).

6. Discussion of Stability

From the results in part 2, we can see that the explicit scheme is unstable for case 2 ($U = 1, \Gamma = 0$) while the implicit and Crank-Nicolson schemes are stable for both cases. This is because without

diffusion, the largest eigenvalue of the coefficient matrix in the explicit scheme will be greater than 1, causing the numerical error to grow exponentially. The inclusion of diffusion in case 1 helps to stabilize the numerical solution by damping out high-frequency oscillations, making the explicit scheme conditionally stable.

For case 1 ($U = 1, \Gamma = 0.01$), the explicit scheme is stable as long as Δt satisfies certain stability conditions. Specifically, $\Delta t \leq \frac{\Delta x^2}{2\Gamma}$ for the diffusion term and $\Delta t \leq \frac{\Delta x}{U}$ for the advection term. For $N = 501$, the spatial step size Δx is approximately 0.08, which means that the stability condition for the diffusion term requires $\Delta t \leq \frac{(0.08)^2}{2 \cdot 0.01} = 0.32$, and the stability condition for the advection term requires $\Delta t \leq \frac{0.08}{1} = 0.08$. Therefore, as long as Δt is less than or equal to 0.08, the explicit scheme will be stable for case 1.

On the other hand, the implicit and Crank-Nicolson schemes are unconditionally stable, meaning they do not have such restrictions on Δt for stability. Therefore, they can handle both cases without becoming unstable.

7. Discussion of Computational Cost

The computational cost of the explicit scheme is generally lower than that of the implicit and Crank-Nicolson schemes because it only requires a single update per time step, while the implicit and Crank-Nicolson schemes require solving a system of equations at each time step. The explicit scheme has a computational complexity of $O(N)$ per time step, while the implicit and Crank-Nicolson schemes have a computational complexity of $O(N^2)$ per time step due to the need to solve a tridiagonal system of equations.

To test this, we can measure the runtime of each scheme for a fixed number of time steps while varying the number of spatial points N. Figure 5 shows a log-log plot of runtime versus N for the three schemes. Specifically, we ran the three schemes for $N = 101, 201, 401, 801, 1601$. Unexpectedly, the runtime of the implicit and Crank-Nicolson schemes scales approximately as $O(N)$ instead of $O(N^2)$. This is likely because we are using the TDMA to solve the tridiagonal system of equations, which has a computational complexity of $O(N)$, rather than a more general solver that would have a computational complexity of $O(N^2)$. The explicit scheme also shows an $O(N)$ scaling as expected. Overall, the results suggest that while the implicit and Crank-Nicolson schemes have a higher computational cost than the explicit scheme, they can still be efficient for large values of N when using an appropriate solver.

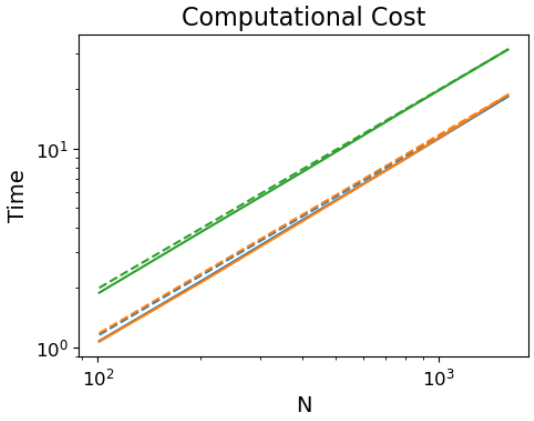

Figure 5: Runtime of the explicit scheme (solid, blue), implicit scheme (solid, orange), and Crank-Nicolson scheme (solid, green) as a function of the number of spatial points N. The slopes of the dashed lines are all 1.0.